Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика, искусственный интеллект и системы управления»

КАФЕДРА «Компьютерные системы и сети»

ОТЧЕТ

по лабораторной работе № 1

по курсу «Технология разработки программных систем»

на тему: «Структуры и методы обработки данных»

Вариант № 11

Студент ИУ6-45Б (Группа) _____ (Подпись, дата) Ионов Н. В. (И. О. Фамилия)

Преподаватель _____ (Подпись, дата) Пугачев Е. К. (И. О. Фамилия)

2026 г.

1 Введение

Цель лабораторной работы: исследование структур данных, методов их обработки и оценки.

Задание.

1. Ознакомиться с теоретическими сведениями об абстрактных структурах данных и методах их обработки.

2. Для указанной задачи и структуры данных предложить способ реализации и определить требуемый объем памяти.

3. Провести анализ заданных методов поиска, упорядочения и корректировки данных. Оценить время выполнения соответствующих операций.

4. Предложить альтернативный вариант решения задачи, в котором должно быть минимум одно улучшение. Улучшения могут касаться как структуры данных, так и основных операций. Обосновать новые решения, используя количественные и качественные критерии.

Задача	Структура данных	Поиск	Упорядовачиние	Корректировка
2	Таблица	Вычисление адреса	Пузырьком	Удаление сдвигом

Таблица 1.1 – Условие исходного задания

Задача.

Дана таблица материальных нормативов, состоящая из K записей фиксированной длины вида: код детали; код материала; единица измерения; номер цеха; норма расхода.

Лабораторная работа будет выполнена на языке `C`.

2 Основная часть

2.1 Реализация

Для хранения таблицы будет использоваться статический массив длины K. Причём элементом этого массива будет запись `data`.

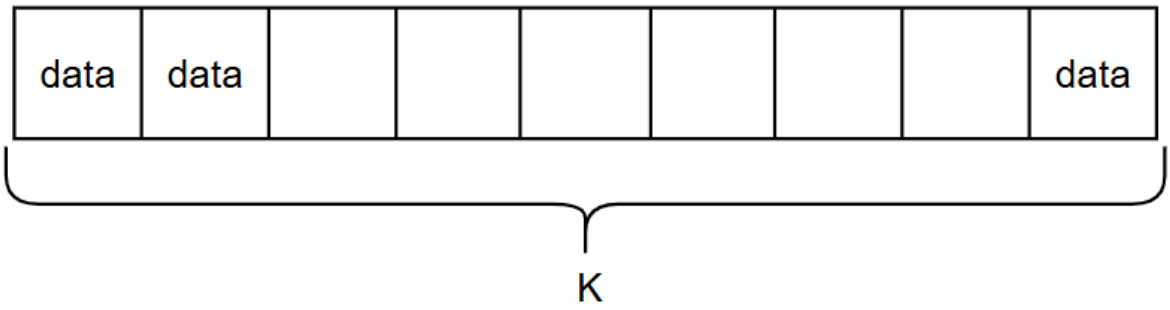

Рисунок 2.1 – Реализация хранения таблицы в виде статического массива

Ниже представлен код, который реализует структуру хранения данных.

Листинг 2.1 – Реализации структуры `data` и соответствующей таблицы

```
#define MAX_LEN_UNIT 5
#define MAX_LEN_MATERIAL 8
#define MAX_LEN_MEASURE 12
#define MAX_LEN_PRODUCTION 8
#define MAX_LEN_E_CON 4
#define k 2000

typedef struct {
    char unit_code[MAX_LEN_UNIT];
    char material_code[MAX_LEN_MATERIAL];
    char measure[MAX_LEN_MEASURE];
    char production_id[MAX_LEN_PRODUCTION];
    char expected_consumption[MAX_LEN_E_CON];
    } data;

data table[k];
```

2.2 Определение объёма данных

- Поскольку все поля — `char`, структура имеет выравнивание 1. Это означает, что её размер равен сумме полей, т.е. 37 байт.
- Объем памяти, занимаемой таблицей, содержащим K записей: $K \times 37$ байт.

2.3 Анализ методов

Сначала рассмотрим алгоритм поиска – `вычисление адреса`.

Листинг 2.2 – Реализации полиномиальной хэш-функции для вычисления адреса

```
int _hash(const char* unit_code, int k) {
            unsigned long hash = 5381;
            int c = 0;
            while ((c = *unit_code++) && c != '\0') {
                hash = ((hash << 5) + hash) + c;
            }
            return hash % k;
}
```

Рассчитаем количество тактов для вычисления адреса. Пусть длина `unit_code` равна L.

$$T_{\texttt{hash}} = t_{\texttt{init}} + t_{\texttt{loop}} + t_{\texttt{return}}$$
$$t_{\texttt{init}} = t_{=} + t_{=} = 2 + 2 = 4$$
$$t_{\texttt{return}} = t_{/} + t_{=} = 28 + 2 = 30$$

В языке `C` компиляторы `gcc, clang` используют ленивые вычисления для оператора `&&`, поэтому поведение цикла зависит от значения прочитанного символа. Рассмотрим два случая.

1. $c \neq 0$

$$T(c = \texttt{*unit_code++}) = t_{\wedge} + t_{++} + t_{=} = 2 + 1 + 2 = 5$$

$$T(\texttt{c \&\& c != '\textbackslash 0'})_{\text{при } c \neq 0} = t_{+} + t_{\text{переход}} + t_{+} + t_{\&\&} = 2 + 1 + 2 + 1 = 6$$

Итого на проверку условия: $5 + 6 = 11$

$$T(\texttt{hash = ((hash « 5) + hash) + c}) = t_{\rightarrow} + t_{+} + t_{+} + t_{=} =$$

$$= 1 + 2 + 2 + 2 = 7$$

$$t_{\texttt{iter}} = 18$$

2. $c = 0$

$$T(c = \texttt{*unit_code++}) = 5$$

$$T(\texttt{c \&\& c != '\textbackslash 0'})_{\text{при } c=0} = t_{+} + t_{\text{переход}} = 2 + 1 = 3$$

Тело цикла не выполняется

$$t_{\texttt{last}} = 8$$

Тогда

$$T_{\texttt{hash}} = t_{\texttt{iter}} \cdot L + t_{\texttt{last}} + t_{\texttt{init}} + t_{\texttt{return}}$$

$$T_{\texttt{hash}} = 18 \cdot L + 42$$

Рассмотрим алгоритм упорядочивания – пузырьком.

Листинг 2.3 – Реализации упорядочения записей в таблице методом пузырьком

```
void buble_sort(data table[], int k) {
    int permutations = -1;
        while (permutations != 0) {
            permutations = 0;
            int i = 0;
            while (i < k-1) {
                if(strcmp(table[i].unit_code,
                   table[i+1].unit_code) > 0){
                    data tmp = table[i];
                    table[i] = table[i+1];
                    table[i+1] = tmp;
                    permutations = 1;
```

```
                }
            ++i;
            }
        }
}
```

Оценка времени упорядочения структуры определяется формулой

$$C_{\texttt{ср}} = \frac{K \cdot (K-1)}{2}$$

Рассмотрим алгоритм корректировки – <u>удаление сдвигом</u>.

Листинг 2.4 – Реализации корректировки записей в таблице методом удаление сдвигом

```
int _remove(data table[], int k, int index) {
    int i = 0;
    while(index + i < k - 1) {
        data tmp = table[index + i];
        table[index+i] = table[index + i + 1];
        table[index + i + 1] = tmp;
        i++;
    }
    return k-1;
}
```

Рассчитаем количество тактов для корректировки.

$$T_{\texttt{remove}} = t_{\texttt{init}} + t_{\texttt{loop}} + t_{\texttt{return}}$$
$$t_{\texttt{init}} = t_{+} = 2$$
$$t_{\texttt{return}} = t_{+} + t_{=} = 2 + 2 = 4$$

Рассмотрим сколько тактов совершается за цикл

$$T(\texttt{index + i < k - 1}) = t_{+} + t_{+} + t_{+} + 1 = 7$$
$$T(\texttt{data tmp = table[index + i]}) = 2$$
$$T(\texttt{table[index+i] = table[index + i + 1]}) = 2$$
$$T(\texttt{table[index + i + 1] = tmp}) = 2$$
$$t_{\texttt{iter}} = 7 + 3 \cdot 2 + 1 = 14$$

Тогда

$$t_{\texttt{loop}} = t_{\texttt{last}} + (k - \text{index} - 1) \cdot t_{\texttt{iter}}$$
$$t_{\texttt{loop}} = 7 + (k - \text{index} - 1) \cdot 14$$
$$T_{\texttt{remove}} = 7 + (k - \text{index} - 1) \cdot 14 + 2 + 4 = 13 + (k - \text{index} - 1) \cdot 14$$

Рассчитаем среднее количество тактов

$$T_{\text{ср}} = \frac{(k - 1 - (k - (k - 1) - 1)) \cdot 14}{2} =$$
$$= \frac{(k - 1) \cdot 14}{2} = (k - 1) \cdot 7 \quad \text{тактов}$$

3 Альтернативная часть

3.1 Предлагаемое улучшение

В качестве альтернативного решения будет использована сбалансированная структура данных – `АВЛ-дерево`. Данная структура позволит улучшить временные характеристики основных операций по сравнению с таблицей.

Операция	**Исходное решение**	**Альтернативное решение**
Поиск	$O(1)$	$O(\log K)$
Упорядочивание	$O(K^2)$	Не требуется
Удаление	$O(K)$	$O(\log K)$
Добавление	—	$O(\log K)$

Таблица 3.1 – Сравнение временных характеристик операций

В АВЛ-дереве данные всегда хранятся в упорядоченном состоянии благодаря инварианту структуры.

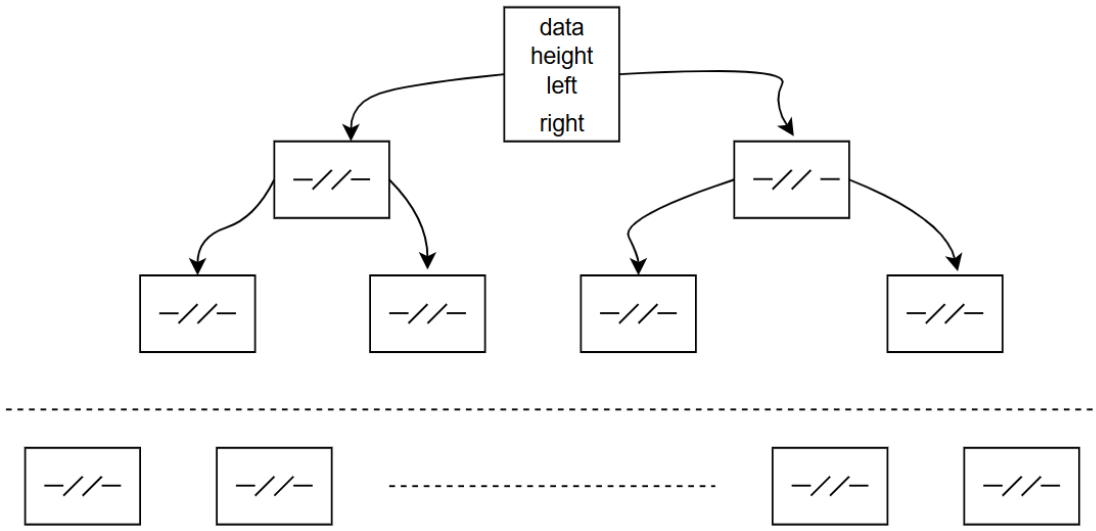

Рисунок 3.1 – Структура дерева

3.2 Структура данных

Для хранения информации о материалах будет использоваться АВЛ-дерево, где ключом является код детали `unit_code`. Предполагается, что

данные загружаются динамически.

Ниже представлена реализация узла дерева:

Листинг 3.1 – Структура узла АВЛ-дерева

```
typedef struct node {
    char unit_code[MAX_LEN_UNIT];
    char material_code[MAX_LEN_MATERIAL];
    char measure[MAX_LEN_MEASURE];
    char production_id[MAX_LEN_PRODUCTION];
    char expected_consumption[MAX_LEN_E_CON];
    int height;
    struct node* left;
    struct node* right;
} node;

node* root = NULL;
```

3.3 Вспомогательные функции

Для работы с АВЛ-деревом необходимы следующие вспомогательные функции:

Листинг 3.2 – Функции для работы с высотой узлов

```
int height(node* x) {
    return x ? x->height : -1;
}

int difference_heights(node* x) {
    return x ? height(x->left) - height(x->right) : 0;
}

void update_height(node* x) {
    int h_left = height(x->left);
    int h_right = height(x->right);
    x->height = (h_left > h_right ? h_left : h_right) + 1;
}
```

3.4 Балансировка дерева

Для поддержания сбалансированности АВЛ-дерева используются малые и большие повороты:

Листинг 3.3 – Малый и большой левый поворот

```
node* rr_rotation(node* x) {
    node* y = x->right;
    x->right = y->left;
    y->left = x;

    update_height(x);
    update_height(y);
    return y;
}

node* rl_rotation(node* x) {
    node* y = x->right;
    node* z = y->left;

    x->right = z->left;
    y->left = z->right;
    z->left = x;
    z->right = y;

    update_height(x);
    update_height(y);
    update_height(z);
    return z;
}
```

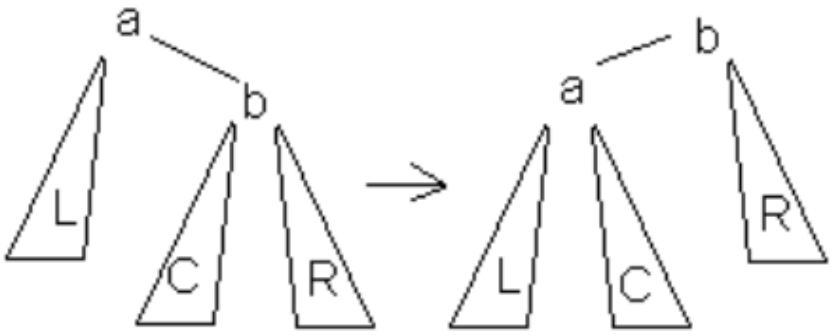

Рисунок 3.2 – Схема малого левого поворота (RR-rotation)

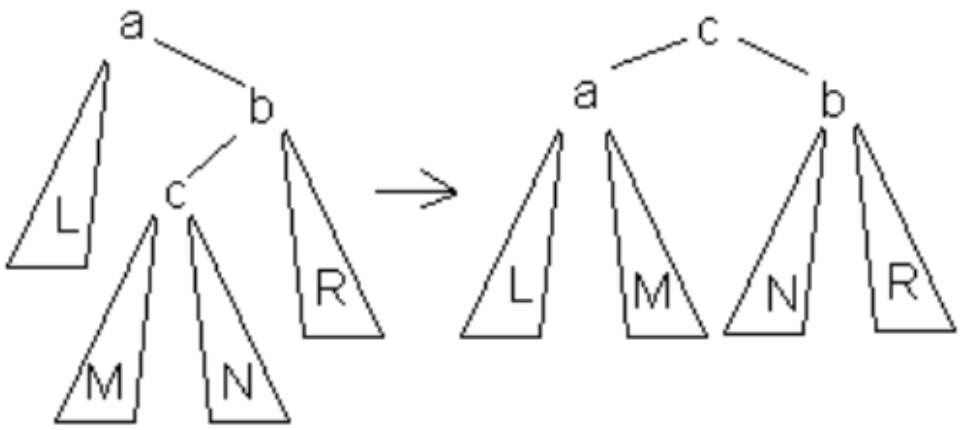

Рисунок 3.3 – Схема большого левого поворота (RL-rotation)

Пояснение левых поворотов, на примере вставки узла

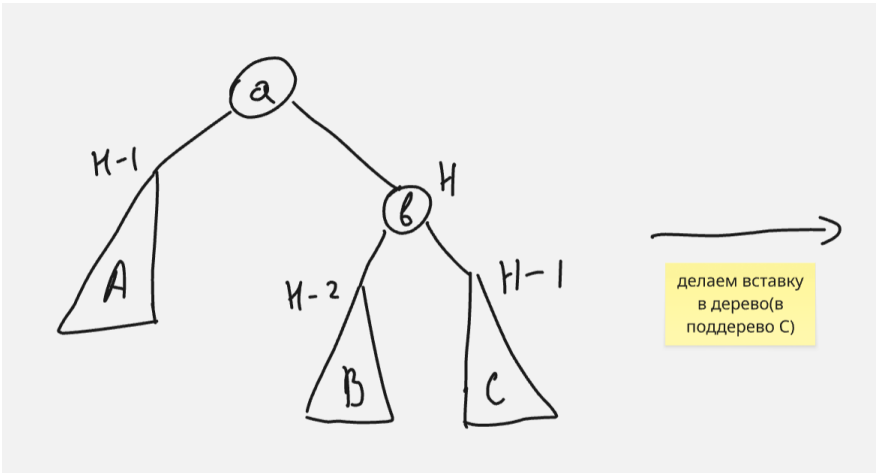

Рисунок 3.4 – Пояснение малого левого поворота (RR-rotation)

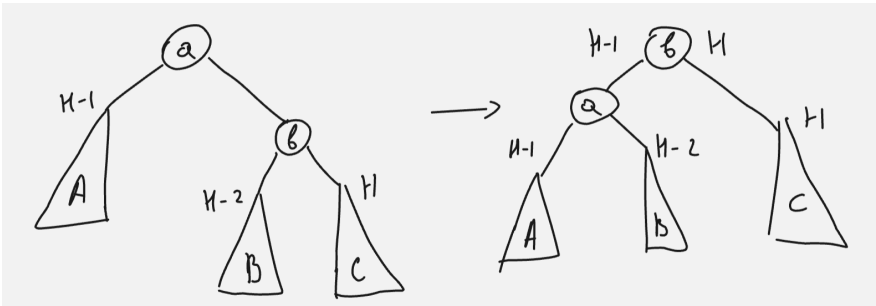

Рисунок 3.5 – Пояснение малого левого поворота (RR-rotation)

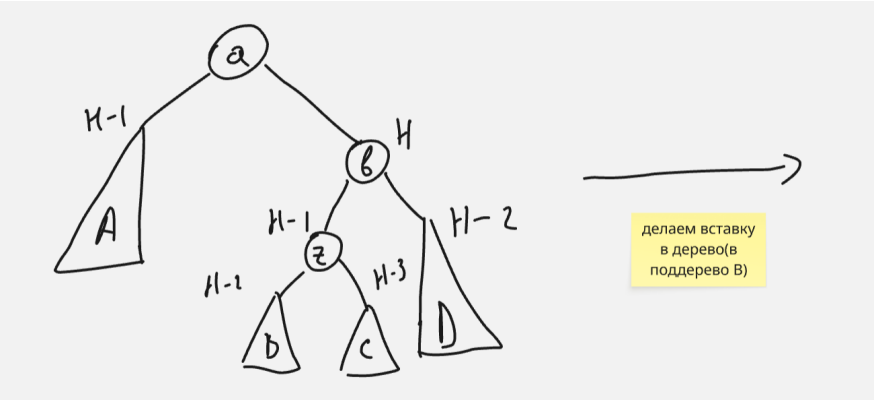

Рисунок 3.6 – Пояснение большого левого поворота (RL-rotation)

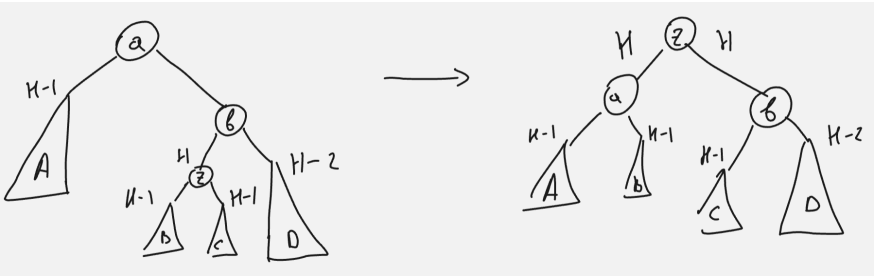

Рисунок 3.7 – Пояснение большого левого поворота (RL-rotation)

Листинг 3.4 – Малый и большой правый поворот

```
node* ll_rotation(node* x) {
    node* y = x->left;
    x->left = y->right;
    y->right = x;

    update_height(x);
    update_height(y);
    return y;
}

node* lr_rotation(node* x) {
    node* y = x->left;
    node* z = y->right;

    x->left = z->right;
    y->right = z->left;
    z->left = y;
    z->right = x;

    update_height(x);
    update_height(y);
    update_height(z);
    return z;
}
```

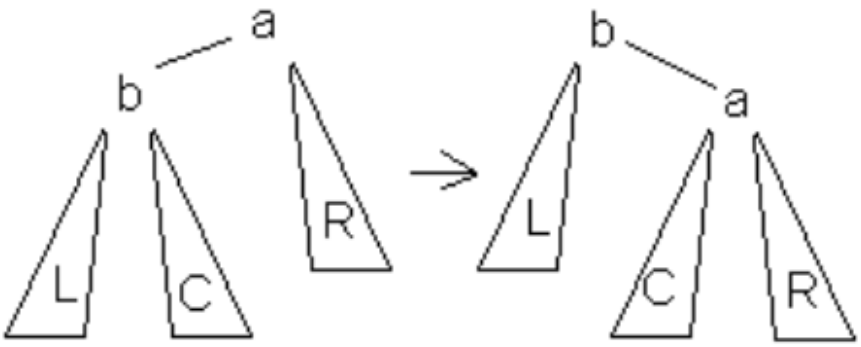

Рисунок 3.8 – Схема малого правого поворота (LL-rotation)

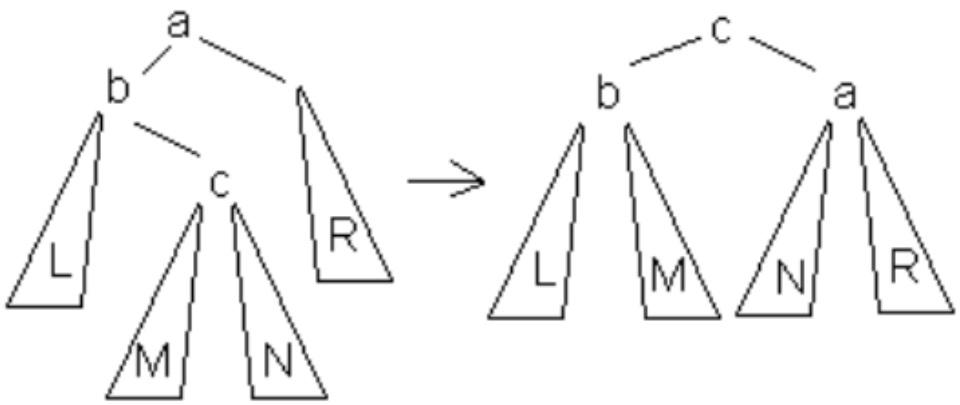

Рисунок 3.9 – Схема большого правого поворота (LR-rotation)

3.5 Операции с деревом

Поиск элемента:

Листинг 3.5 – Функция поиска узла по коду детали

```
node* find(node* x, const char* unit_code) {
    if(x == NULL)
        return NULL;

    int cmp = strcmp(unit_code, x->unit_code);

    if(cmp == 0)
        return x;

    if(cmp < 0)
        return find(x->left, unit_code);
    else
        return find(x->right, unit_code);
}
```

Вставка элемента:

Листинг 3.6 – Функция вставки узла в дерево

```
node* insert(node* x, node* new_node) {
    if (x == NULL)
        return new_node;

    int cmp = strcmp(new_node->unit_code, x->unit_code);

    if (cmp < 0)
        x->left = insert(x->left, new_node);
```

```
    else if (cmp > 0)
        x->right = insert(x->right, new_node);
     else {
        *x = *new_node;
        free(new_node);
        return x;
    }

    update_height(x);
    int balance = difference_heights(x);

    if (balance > 1 && strcmp(new_node->unit_code,
       x->left->unit_code) < 0)
        return ll_rotation(x);

    if (balance < -1 && strcmp(new_node->unit_code,
       x->right->unit_code) > 0)
            return rr_rotation(x);

    if (balance > 1 && strcmp(new_node->unit_code,
       x->left->unit_code) > 0)
            return lr_rotation(x);

    if (balance < -1 && strcmp(new_node->unit_code,
       x->right->unit_code) < 0)
            return rl_rotation(x);

    return x;
}
```

Рассмотрим пример вставки в АВЛ-дерево, но в качестве ключей будут целые числа. При удаления будет такая же структура узла, для наглядности.

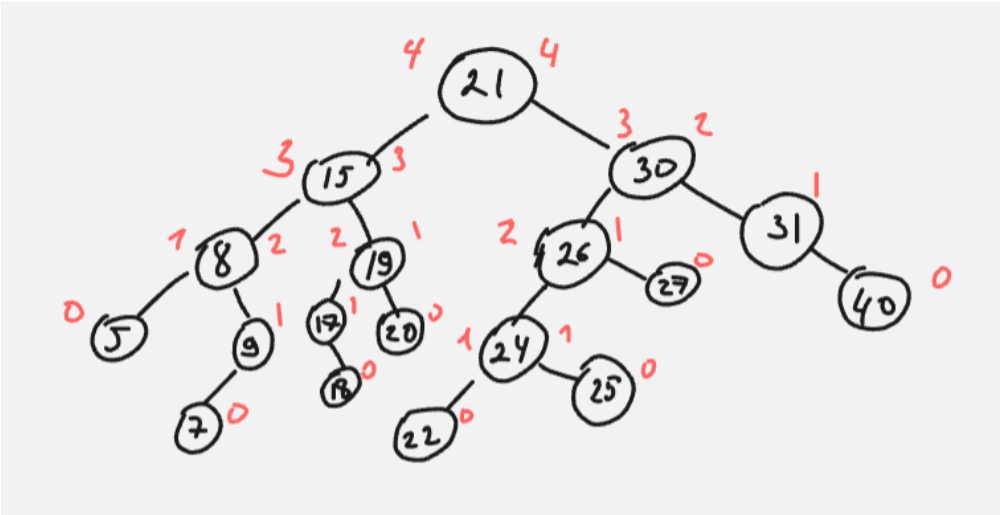

Рисунок 3.10 – Исходное дерево

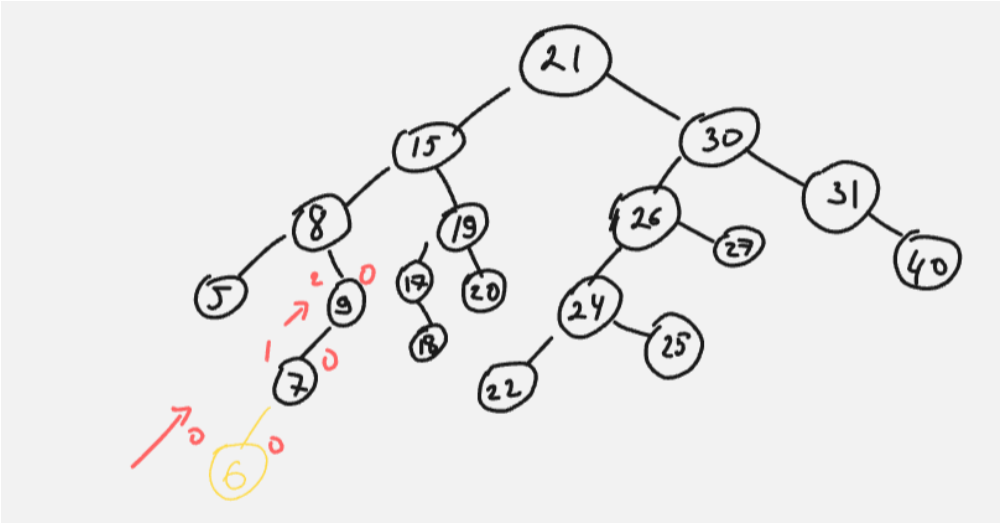

Рисунок 3.11 – Пояснение вставки в самобалансирующиеся дерево

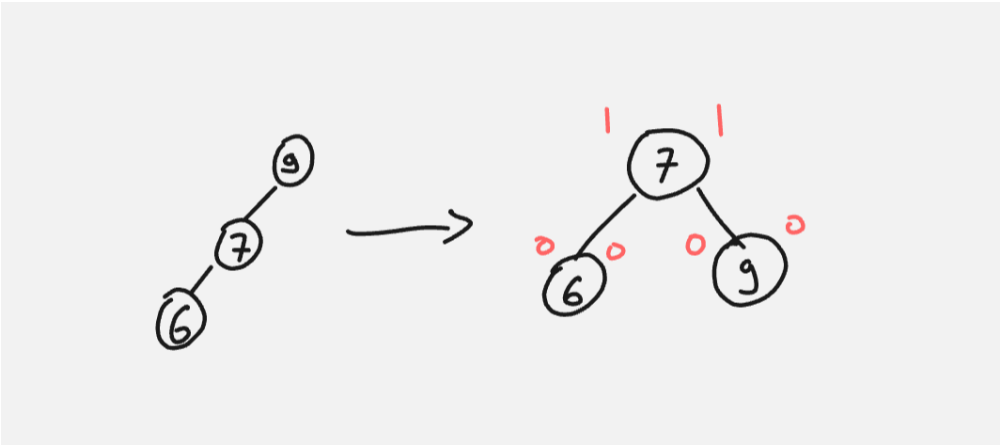

Рисунок 3.12 – Пояснение вставки в самобалансирующиеся дерево

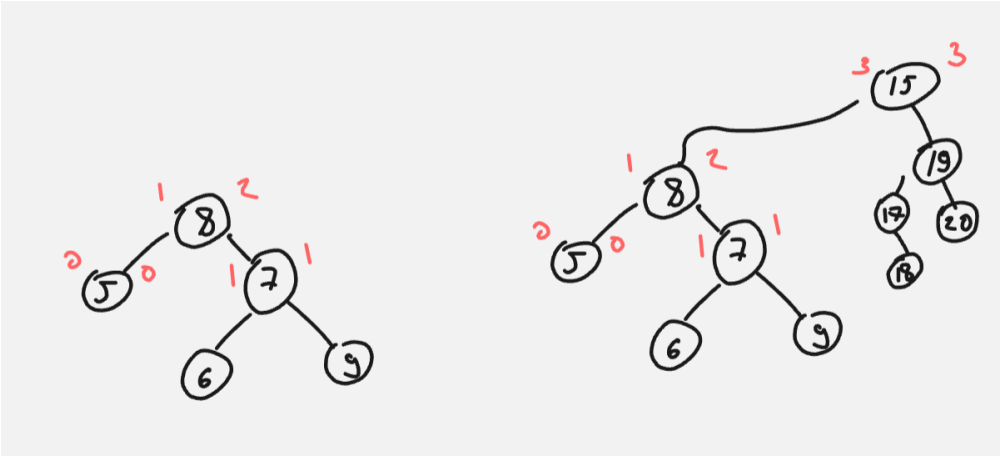

Рисунок 3.13 – Пояснение вставки в самобалансирующиеся дерево

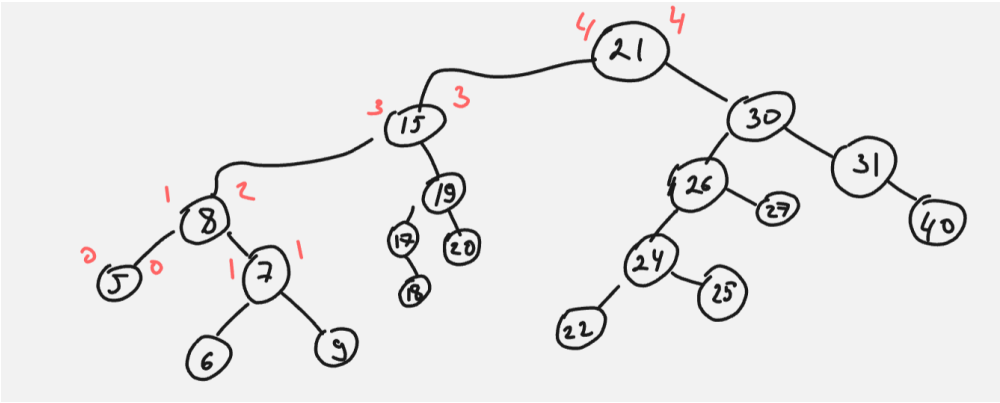

Рисунок 3.14 – Пояснение вставки в самобалансирующиеся дерево

Удаление элемента:

Листинг 3.7 – Функция удаления узла из дерева

```
node* find_min(node* x) {
    while (x->left != NULL)
        x = x->left;
    return x;
}

node* remove(node* x, const char* unit_code) {
    if (x == NULL)
        return NULL;

    int cmp = strcmp(unit_code, x->unit_code);

    if (cmp < 0)
        x->left = remove(x->left, unit_code);
    else if (cmp > 0)
        x->right = remove(x->right, unit_code);
    else {
        if (x->left == NULL || x->right == NULL) {
            node* temp = x->left ? x->left : x->right;
            if (temp == NULL) {
                free(x);
                return NULL;
            } else {
                *x = *temp;
                free(temp);
            }
        }else {
            node* min_right = find_min(x->right);
                strcpy(x->unit_code, min_right->unit_code);
                strcpy(x->material_code,
                   min_right->material_code);
                strcpy(x->measure, min_right->measure);
                strcpy(x->production_id,
                   min_right->production_id);
                strcpy(x->expected_consumption,
                   min_right->expected_consumption);
                x->height = min_right->height;
                x->right = remove(x->right,
                   min_right->unit_code);
```

```
            }
        }

    if (x == NULL)
        return NULL;

    update_height(x);
    int balance = difference_heights(x);

    if (balance > 1 && difference_heights(x->left) >= 0)
        return ll_rotation(x);

    if (balance > 1 && difference_heights(x->left) < 0)
        return lr_rotation(x);

    if (balance < -1 && difference_heights(x->right) <= 0)
        return rr_rotation(x);

    if (balance < -1 && difference_heights(x->right) > 0)
        return rl_rotation(x);

    return x;
}
```

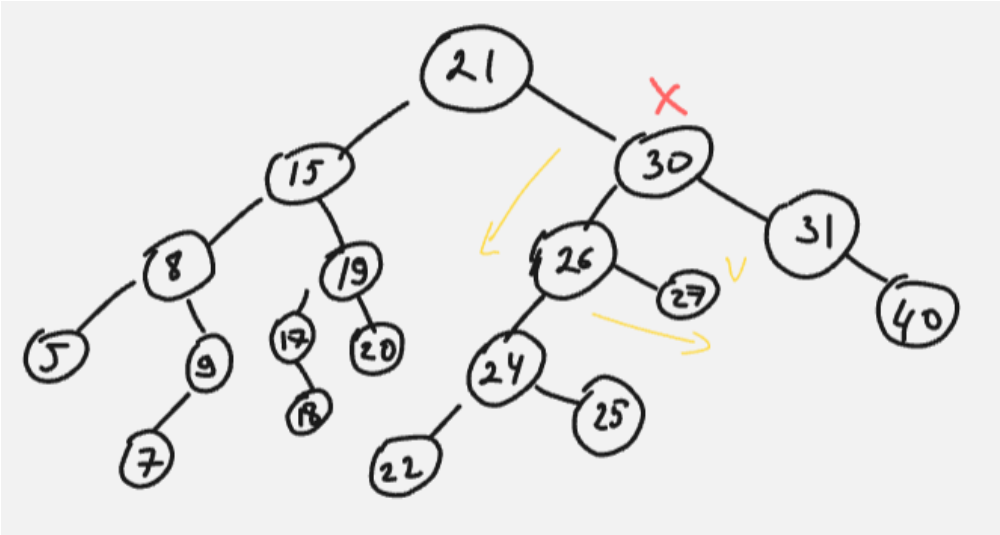

Рисунок 3.15 – Пояснение удаления из самобалансирующегося дерева

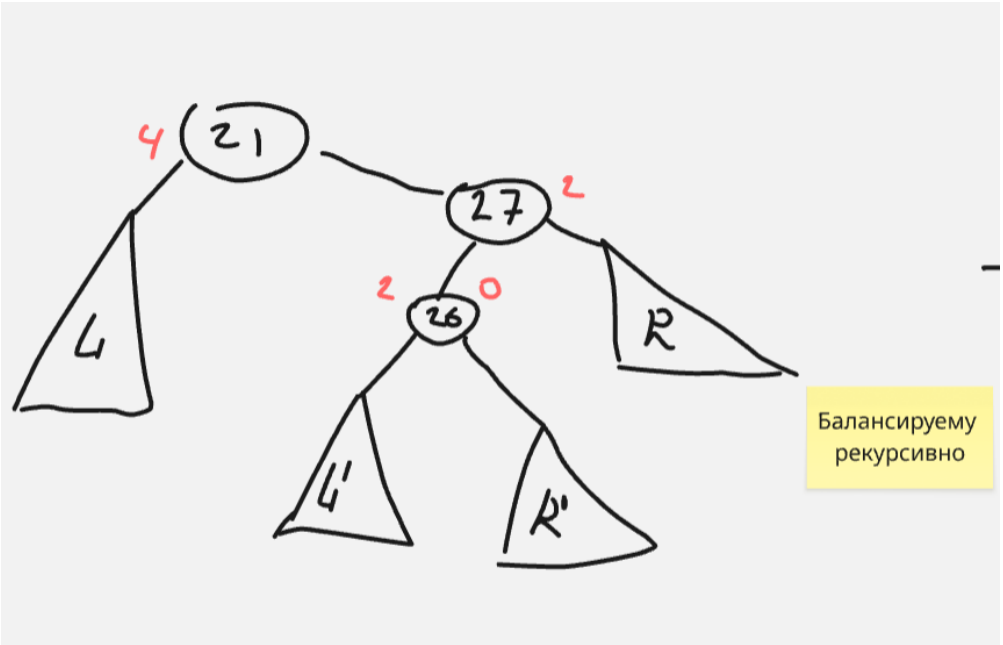

Рисунок 3.16 – Пояснение удаления из самобалансирующегося дерева

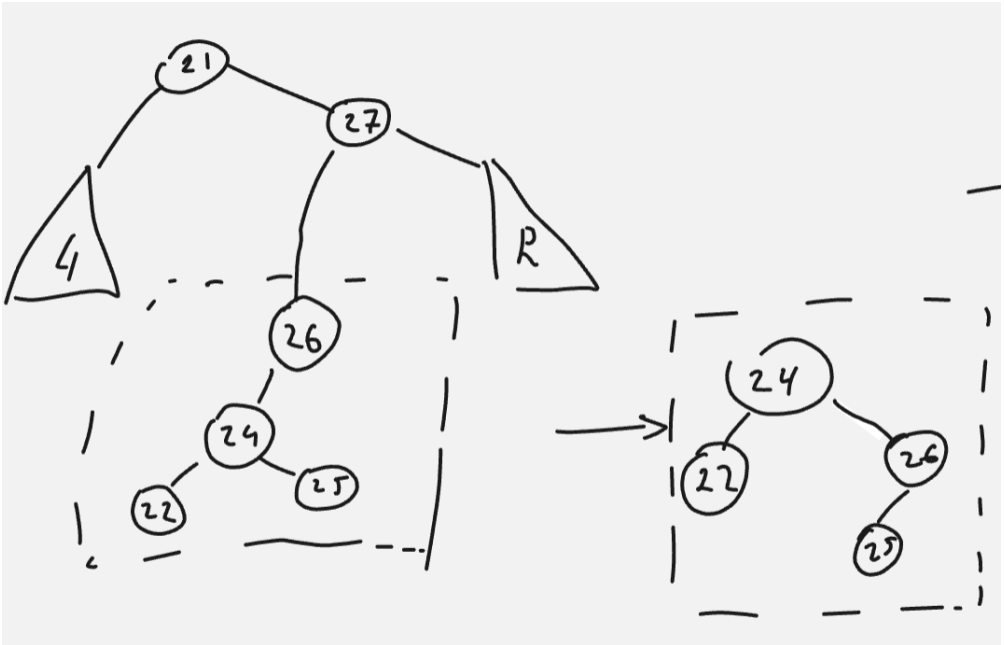

Рисунок 3.17 – Пояснение удаления из самобалансирующегося дерева

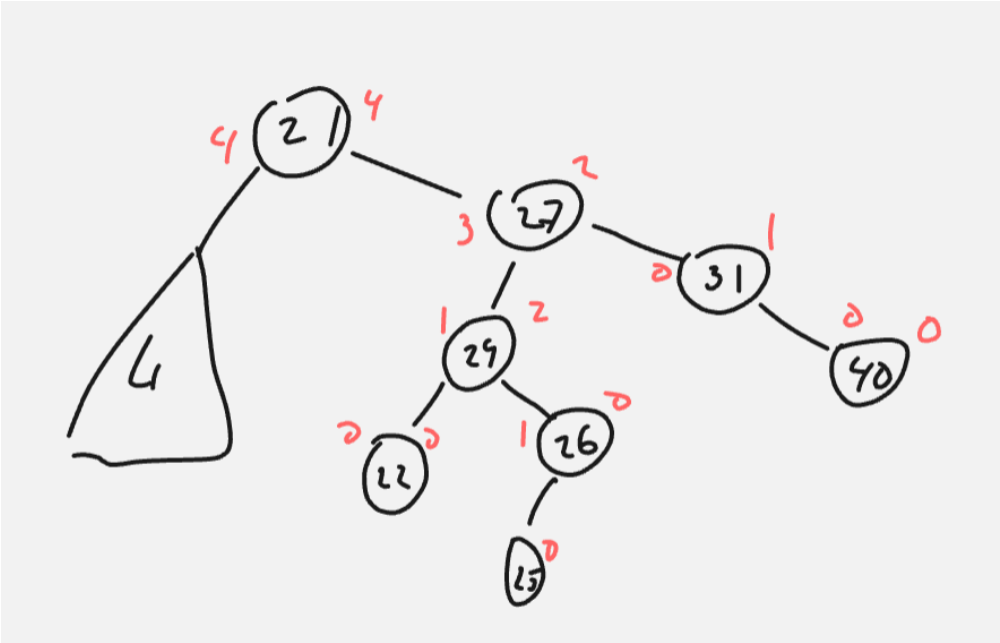

Рисунок 3.18 – Пояснение удаления из самобалансирующегося дерева

3.6 Оценка эффективности

Использование АВЛ-дерева вместо таблицы со статическим массивом даёт следующие преимущества:

- **Динамическое расширение:** отсутствие ограничения на максимальное количество записей K
- **Гарантированное время поиска:** $O(\log K)$ вместо ожидаемого $O(1)$ при хэшировании
- **Эффективная корректировка:** $O(\log K)$ против $O(K)$ при удалении сдвигом
- **Автоматическая упорядоченность:** данные всегда отсортированы по ключу

Недостатком является дополнительная память на хранение указателей и высоты узлов:

- Для K узлов требуется дополнительно $K \cdot (2 \cdot 8 + 4)$ байт
- Временные затраты на балансировку при вставке и удалении

Таким образом, АВЛ-дерево является эффективной альтернативой для динамически изменяемых данных с частыми операциями поиска и корректировки.

4 Заключение

	Структура данных	**Алгоритм поиска**	**Алгоритм упорядочивания**	**Корректировка**
Исходный вариант	Статический массив	Вычисление адреса	Пузырьком	Удаление сдвигом
	$37K$ байт	$18L + 42$ тактов	$\frac{K(K-1)}{2}$ среднее кол-во перестановок	$(K-1) \cdot 7$ среднее кол-во тактов
Альтернативный вариант	АВЛ-дерево	Бинарный поиск	Не требуется	Удаление с балансировкой
	$57K$ байт	$\log_2 K$ среднее кол-во сравнений	Постоянно благодаря структуре дерева	$2 \cdot \log_2 K$ операций

Таблица 4.1 – Сравнительный анализ исходного и альтернативного решения

Примечание: K — количество записей, L — длина кода детали, i — индекс удаляемого элемента.

Анализ недостатков исходного решения

При детальном рассмотрении метода вычисления адреса были выявлены следующие существенные недостатки:

- **Наличие пустых ячеек:** Метод вычисления адреса редко дает идеальное распределение без коллизий. На практике массив таблицы содержит незаполненные ячейки, что приводит к неэффективному использованию памяти.

- **Коллизии:** При совпадении хэш-кодов для разных ключей возникает необходимость разрешения коллизий. Это может привести к значительному увеличению времени поиска.

- **Статический размер:** При переполнении таблицы требуется её полное перестроение.

Преимущества альтернативного решения

- **Динамическое расширение:** Отсутствие ограничения на максимальное количество записей; память выделяется по мере необходимости.

- **Гарантированная производительность:** Время выполнения основных операций (поиск, вставка, удаление) гарантированно составляет $O(\log K)$ в худшем случае.

- **Автоматическая упорядоченность:** Данные всегда хранятся в отсортированном виде по ключу, что упрощает выполнение операций вывода и поиска диапазонов. Операция упорядочивания не требуется.

- **Эффективная корректировка:** Удаление и добавление записей выполняется за логарифмическое время.

Недостатки альтернативного решения

- **Дополнительные накладные расходы по памяти:** На каждый узел требуются указатели на левое и правое поддеревья ($2 \cdot 8$ байт) и хранение высоты, что при больших K приводит к увеличению потребляемой памяти по сравнению с массивом.

- **Сложность реализации:** Поддержка баланса требует реализации механизма поворотов.

- **Константное время:** Несмотря на логарифмическую асимптотику, операции с АВЛ-деревом имеют более высокие константные затраты, чем простое обращение по индексу в массиве.

Таким образом, выбор между статической таблицей и АВЛ-деревом зависит от условий эксплуатации. Если данные статичны, известны заранее и критически важна максимальная скорость доступа при минимальных накладных расходах — предпочтительнее использовать таблицу с функцией

поиска - вычисление адреса. Если же данные динамически изменяются, требуется гарантированное время отклика и поддержка упорядоченности, то АВЛ-дерево является более эффективным и надежным решением.